C++ COMPLETE REFERENCE LIBRARY • MODULE 02

02 DATA TYPES AND VARIABLES — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 fundamental types and representations
3. **Theory Chapter:** 02 variable initialization and narrowing
4. **Theory Chapter:** 03 const constexpr and auto
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

02 — Data Types and Variables

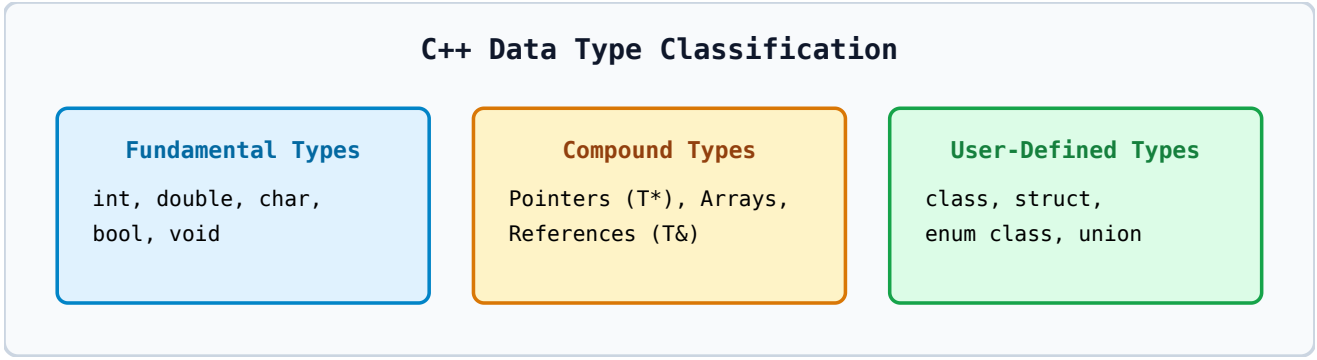
This module covers C++ fundamental types (`bool` , `char` , `int` , `long long` , `float` , `double`), signed vs. unsigned representation, brace initialization (`int x{10};`), `const` vs. `constexpr` , and type deduction (`auto`).

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_fundamental_types_and_representations.md` (./01_THEORY/01_fundamental_types_and_representations.md)
  - `02_variable_initialization_and_narrowing.md` (./01_THEORY/02_variable_initialization_and_narrowing.md)
  - `03_const_constexpr_and_auto.md` (./01_THEORY/03_const_constexpr_and_auto.md)
- **Programs (`02_PROGRAMS/`)**:
  - `01_fundamental_types_and_sizes.cpp` (./02_PROGRAMS/01_fundamental_types_and_sizes.cpp) (Category A)

  - `02_const_and_constexpr.cpp` (./02_PROGRAMS/02_const_and_constexpr.cpp) (Category A)
  - `03_narrowing_conversion_compile_error.cpp` (./02_PROGRAMS/03_narrowing_conversion_compile_error.cpp) (Category B)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_data_type_hierarchy.svg` (./05_DIAGRAMS/diagram_01_data_type_hierarchy.svg)

01 — Fundamental Data Types and Two's Complement Representation



Data Type Hierarchy

1. Standard Minimum Bit Widths (`[basic.fundamental]`)

- `bool` : 1 byte (`true` or `false`).
- `char` : Guaranteed `sizeof(char) == 1` byte (at least 8 bits). Note: Whether plain `char` is signed or unsigned is **implementation-defined**!
- `short` : At least 16 bits (typically 2 bytes).
- `int` : At least 16 bits (**4 bytes / 32 bits** on all modern 32/64-bit desktop/server platforms, range -2^{31} to $2^{31}-1$).
- `long long` : At least 64 bits (8 bytes).
- `double` : 64-bit IEEE-754 floating-point (~15–17 decimal digits of precision).

02 — Variable Initialization Forms and Narrowing Prevention

1. **Copy Initialization**: `int x = 10;`
2. **Direct Initialization**: `int x(10);`
3. **Direct-List / Brace Initialization (C++11–C++23)**: `int x{10};` or `int x{};` (zero-initialized to `0`).
   - **Key Advantage of `{}`**: Prohibits **narrowing conversions** at compile time (`int x{3.14};` is a **Compile-Time Error**, whereas `int x = 3.14;` silently truncates `.14`!).
4. **Uninitialized Local Variables**: Reading an uninitialized local non-static variable (`int x; cout << x;`) is **Undefined Behavior**.

03 — `const`, `constexpr`, and `auto` Type Deduction

- `const`: Promises that the variable cannot be modified after initialization (value may be computed at runtime or compile time).
- `constexpr` **(C++11–C++23)**: Guarantees that the variable's value is a **compile-time constant expression** (usable as an array bound `int arr[MAX];`).
- `auto`: Deduces the variable's type at compile time from its initializer (`auto pi = 3.14;` deduces `double`).

Visual Concept & Memory Diagrams (02 DATA TYPES AND VARIABLES)

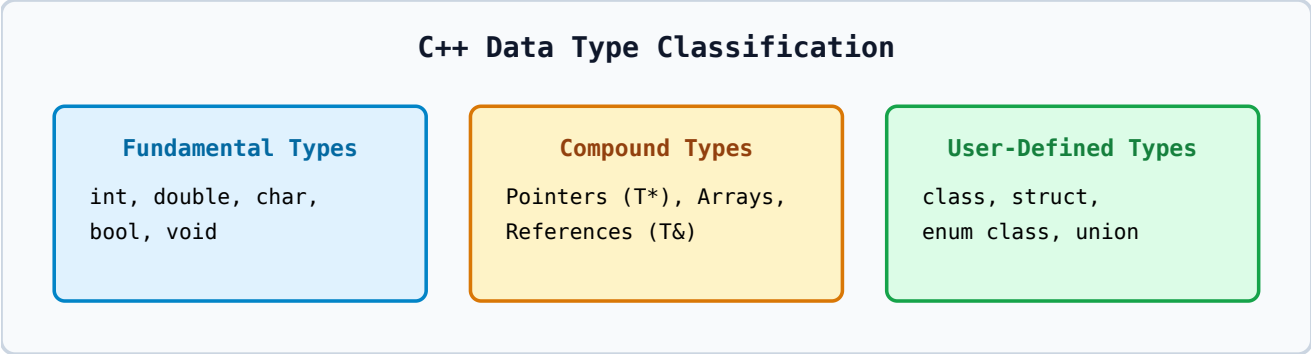


Diagram: diagram 01 data type hierarchy

Annotated C++ Programs (02 DATA TYPES AND VARIABLES)

Program: `01_fundamental_types_and_sizes.cpp`

C++ SOURCE FILE • 01_fundamental_types_and_sizes.cpp

```cpp
// Program Title: Fundamental Data Types, Initialization, and Value Ranges
// Topic: Data Types and Variables
// Purpose: Demonstrate initializing fundamental C++ types and printing their values.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_fundamental_types_and_representations.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    int count{42};
    double ratio{3.5};
    char grade{'A'};
    bool passed{true};

    cout << "count: " << count << ", ratio: " << ratio
         << ", grade: " << grade << ", passed: " << passed << endl;
    return 0;
}

/*
Expected Deterministic Output:
count: 42, ratio: 3.5, grade: A, passed: 1
*/
```

Program: `02_const_and_constexpr.cpp`

C++ SOURCE FILE • 02_const_and_constexpr.cpp

```cpp
// Program Title: Compile-Time Constants Using constexpr and const
// Topic: Data Types and Variables — const and constexpr
// Purpose: Use a constexpr constant as a compile-time array size bound.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_const_constexpr_and_auto.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    constexpr int SIZE = 4;
    int squares[SIZE] = {1, 4, 9, 16};

    for (int i = 0; i < SIZE; i++) {
        cout << squares[i] << " ";
    }
    cout << endl;
    return 0;
}

/*
Expected Deterministic Output:
1 4 9 16
*/
```

Program: `03_narrowing_conversion_compile_error.cpp`

C++ SOURCE FILE • 03_narrowing_conversion_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Narrowing Conversion Rejected by Brace Initialization
// Topic: Data Types and Variables — List Initialization Rules
// Purpose: Show how brace initialization {3.14} catches floating-to-int narrowing at compile time.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/02_variable_initialization_and_narrowing.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

int main() {
    int x{3.14}; // COMPILE ERROR: narrowing conversion of '3.14' from 'double' to 'int'
    cout << x << endl;
    return 0;
}

/*
Why Compilation Fails:
ISO C++ [dcl.init.list] forbids narrowing conversions inside brace-init-lists '{}'.
Use a matching type 'double x{3.14};' or an explicit cast 'int x{static_cast<int>(3.14)};'.
*/
```

Conceptual Questions — Module 02: Data Types and Variables

Q1. Distinguish between `const` and `constexpr` in C++. **Q2.** Why does brace initialization (`int x{3.9};`) prevent bugs that `int x = 3.9;` allows?

Debugging Questions — Module 02

Q4. Find the bug in `int sum; for (int i = 1; i ⩽ 5; i++) sum += i;`.

Predict the Output Questions — Module 02

Q3. Predict the output of `unsigned int u = 0; u = u - 1;` vs signed overflow.

Programming Questions — Module 02

Q5. Write a C++23 program that converts temperature from Celsius to Fahrenheit using `double` precision.

Detailed Solutions — Conceptual Questions (Module 02)

- **Q1 Solution**: `const` means read-only after initialization (may be initialized at runtime); `constexpr` guarantees compile-time evaluation.
- **Q2 Solution**: `{}` list-initialization makes narrowing conversions (`double` to `int`) ill-formed at compile time instead of silently truncating fractional digits.

Detailed Solutions — Predict the Output & Debugging (Module 02)

- **Q3 Solution**: Unsigned integers obey well-defined modulo 2^N arithmetic (`0U - 1U` wraps to `UINT_MAX`), whereas signed integer overflow is Undefined Behavior.
- **Q4 Solution**: `int sum;` is uninitialized (contains indeterminate value), so `sum += i;` has Undefined Behavior! Initialize `int sum = 0;`.

Detailed Solutions — Programming Questions (Module 02)

CPP

```cpp
#include <iostream>
using namespace std;
int main() {
    double c = 25.0;
    double f = (c * 9.0 / 5.0) + 32.0;
    cout << "Fahrenheit: " << f << endl;
    return 0;
}
// Expected Output: Fahrenheit: 77
```